

Causality - SoSe 2025

Constraint-Based Causal Discovery

Alexander Marx

Last lecture, we discussed the following concepts

- Markov Equivalence
- Faithfulness
- Causal Minimality
- Causal Sufficiency
- Markov Blankets
- Grow and Shrink Algorithm

Definition Given a DAG $\mathcal{G} = (\mathbf{V}, \mathcal{E})$, and a distribution P over $\mathbf{X}$ with density p .

1. P is said to be **faithful** to $\mathcal{G}$ if for all pairwise disjoint subsets $\mathbf{A}, \mathbf{B}, \mathbf{C} \subseteq \mathbf{V}$:

$$\mathbf{X}_{\mathbf{A}} \perp\!\!\!\perp \mathbf{X}_{\mathbf{B}} \mid \mathbf{X}_{\mathbf{C}} \implies \mathbf{A} \perp\!\!\!\perp_{\mathcal{G}} \mathbf{B} \mid \mathbf{C} .$$

2. P satisfies **causal minimality** wrt $\mathcal{G}$ if it is Markovian with respect to $\mathcal{G}$, but not to any proper subgraph $\mathcal{G}' = (\mathbf{V}, \mathcal{E}')$ of $\mathcal{G}$ with $\mathcal{E}' \subset \mathcal{E}$.

Alternatively, if $\mathcal{G}$ is an **I-map** to P but no proper subgraph $\mathcal{G}'$ of $\mathcal{G}$ is an I-map to P , then P **satisfies causal minimality** wrt $\mathcal{G}$. Recap difference minimality and faithfulness

Algorithm 1: GS for Markov Blankets

input : $\mathbf{X}_V$ with joint P , Target $t \in V$

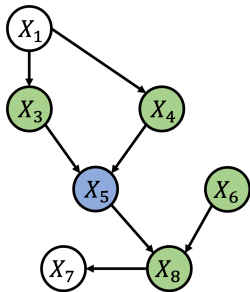
output : $\text{MB}(t)$

```
1  $\mathbf{S} \leftarrow \emptyset$ ;  
  // Grow Phase  
2 while  $(\exists k \in V \setminus \{t\} : X_t \not\perp\!\!\!\perp X_k \mid \mathbf{X}_S)$  do  
3    $\mathbf{S} \leftarrow \mathbf{S} \cup \{k\}$  ;  
  // Shrink Phase  
4 while  $\exists k \in \mathbf{S} : X_t \perp\!\!\!\perp X_k \mid \mathbf{X}_{S \setminus \{k\}}$  do  
5    $\mathbf{S} \leftarrow \mathbf{S} \setminus \{k\}$  ;  
6 return  $\mathbf{S}$ 
```

Example Run ($t = 5$):

SP: $X_5 \perp\!\!\!\perp X_1 \mid \{X_3, X_4, X_6, X_8\}$

$\mathbf{S} = \{3, 4, 6, 8\} = \text{MB}(5)$



P is faithful and Markov wrt $\mathcal{G}$.

Outline

In today's lecture, we will dive deeper into causal discovery, and

- Prove the correctness of GS
- Give an overview to Causal Discovery Frameworks
- Learn about the SGS and the PC algorithm

Theorem Let $(\mathcal{G}, P)$ be a **DAG model**, where P is **faithful** wrt $\mathcal{G}$. Further, assume access to a **perfect (conditional) independence test**. For any target node $t \in \mathbf{V}$, the Grow and Shrink algorithm (Algorithm 1) returns the **correct** Markov blanket $\text{MB}(t)$.

Proof Outline

1. Show that after the grow phase, $\mathbf{S}$ is a superset of $\text{MB}(t)$
2. Prove that in the shrink phase:
 - i. Any node $k \in \mathbf{S}$ and in $\text{MB}(t)$ is kept in $\mathbf{S}$
 - ii. Any node $k \in \mathbf{S}$ and not in $\text{MB}(t)$ is removed from $\mathbf{S}$

A **proof** is provided in the notes for Lecture 8.

How can we learn more than just the local neighbourhood?

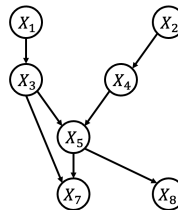
Given Observational Data

Causal Graph

X_1 X_2 X_3 X_4 X_5 X_6 X_7 X_8

```
-1.164394838 0.1315188938 1.3882313792 0.2168386644 2.38448261 -0.805881632 0.187786684 -0.189548688
-1.395158481 0.8733378925 2.0417825902 0.446978484 1.98577622 -0.1809972482 0.784875889 1.212846886
0.609429252 0.6834298314 -0.2833519157 0.386077235 1.29439609 0.1490988801 -1.08795622 0.4198821548
0.181768913 -0.1373663637 -1.7277215668 -1.53098385 0.91919197 -0.8305614882 2.811493628 0.016441312
0.836398818 -1.1846486652 -0.3518461518 -0.2873648491 -0.88412612 -0.8686619115 -0.849795182 0.708183883
-0.22882832 1.1234724951 -0.2872819203 -0.431887768 -0.82276688 -0.2215518488 -0.8548877083 -0.3329198858
0.466178875 0.3544657787 0.2897212379 0.971082513 1.63714998 0.826122327 -1.095704448 0.4784984888
0.222656134 -0.1243857807 1.4953958608 1.718805783 1.648215640 -0.6987283183 -0.354651338 -0.850265515
0.386082144 -1.1541573626 2.6688912503 -1.2864964379 2.51575751 -0.8611957631 -0.843924815 -0.805848612
0.483298784 1.1946115218 1.4688212514 2.841521194 0.43289161 -0.8878714892 0.778889022 -0.817972379
0.415889188 1.1882178819 0.8767886586 0.53099942 1.87461615 0.896862977 0.88611202 0.237433051
1.478342379 0.7213590886 2.5454933617 0.826127748 0.26365223 -0.8407818866 0.323861271 -0.255484961
0.508722218 -0.3948801084 0.7791182279 1.31609846 0.74875086 -0.8566661217 -0.284769884 0.738176438
0.761588283 -0.4719766882 -0.6198158801 0.258787748 2.48376678 -0.880863847 -0.484871683 1.565197138
0.643709728 0.8982111134 0.8738783188 0.612438844 1.77387780 -0.137621278 1.358717718 -2.09758586
0.887388121 0.8882870833 0.5848248156 0.1313212887 -0.76746028 -0.8674821292 -0.832409227 0.167215867
1.218151746 0.1884627111 0.1481514808 0.509161753 1.835848093 -0.1158838590 -1.845238321 -0.808165671
1.3561768215 -0.4449765574 1.3868352127 2.445791637 1.58476134 0.4516089849 -0.764884839 0.1336128502
0.371778971 -1.3848888714 0.5615788964 0.91314888 1.15874892 0.8528514647 -0.434513089 0.981795864
-0.175147111 0.8715881183 -0.6933911665 -0.56054818 1.33352564 0.8284327577 -1.277588852 0.767495178
-0.416428877 -1.5282621777 -0.9194432637 -1.358884151 1.87161699 -0.821358880 -0.831884116 0.828829556
-2.482631683 -1.121861268 0.4784815696 0.499789284 2.78448718 -0.686272142 -0.81385615 0.287864887
-1.286234829 -0.837674918 0.2844381933 -0.921884635 0.48523532 0.897088498 -1.821862025 -0.839747329
0.21677289 0.2411158578 -0.6574711608 -0.876584842 -1.33811567 -0.1918844957 -0.59979088 -1.057514294
1.664721144 -0.2384889217 -0.687212823 0.258881216 1.78277812 -0.861614682 -0.35192727 -0.768284646
-0.436376472 -0.237817982 0.7737148998 0.462677627 -0.11289683 -0.1959842972 0.483491828 -0.368856242
-0.185488386 -0.3857819138 0.7187142587 0.858481781 -0.08488584 0.087134364 -0.585784868 -1.17337841
0.861829232 -0.251954832 0.1558888561 1.817038478 0.85788256 -0.8182828187 1.885174728 -1.273242413
-0.360573113 -0.8438100578 0.5128774812 0.167554378 -0.52161636 -0.0801118871 -1.131763233 1.802024164
-0.495789264 -0.1385178298 -0.1688475807 -0.8483565448 2.93785287 -0.8628878757 -0.519661673 1.386988363
0.768148264 0.1847881871 0.4678582571 0.30741476 -1.97871828 0.5018481828 -0.843481287 -1.718571175
```

Learn



Causal Discovery: Overview

Constraint-Based

- Measure **conditional independencies**
- Search for graph which **satisfies** them

Examples

SGS, PC

Identifies

CPDAG

(Markov equivalence class)

Score-Based

- **Score** models based on data fit
- Large search space, greedy approach

Examples

GES

Identifies

CPDAG

(Markov equivalence class)

Functional/Restricted

- **Additional assumptions** on model class, noise distribution, etc.

Examples

LiNGAM, RESIT, CAM

Identifies

DAG

(Usually: True model)

Causal Discovery: Overview

Constraint-Based

- Measure **conditional independencies**
- Search for graph which **satisfies** them

Examples

SGS, PC

Identifies

CPDAG

(Markov equivalence class)

Score-Based

- **Score** models based on data fit
- Large search space, greedy approach

Examples

GES

Identifies

CPDAG

(Markov equivalence class)

Functional/Restricted

- **Additional assumptions** on model class, noise distribution, etc.

Examples

LiNGAM, RESIT, CAM

Identifies

DAG

(Usually: True model)

Question: Given iid data, which tasks are needed to learn a causal network that go beyond traditional regression or feature selection?

- **Distinguish adjacent nodes** from the ones only connected through a path that contains at least one intermediate node
 - In regression, e.g., we are interested in all variables that improve the fit, which may include non-adjacent nodes
- Determine the **edge directions**
 - In regression, e.g., we do not distinguish children and parents

Key idea: For any (causal) Bayesian network $(\mathcal{G}, P)$:

- **Adjacent nodes** cannot be d-separated
- For **non-adjacent nodes**, we can find a separating set

If P is **faithful** to $\mathcal{G}$ then the above also holds for independence.

SGS Algorithm

The **SGS algorithm** (named after the authors Spirtes, Glymour and Scheines, 1990) consists of three phases:

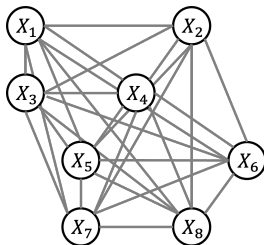
1. **Skeleton**: Finds the causal skeleton by exploiting independencies
2. **V-Structure**: Orients all v-structures based on the skeleton, i.e., checks all **unshielded triples** $X - Y - Z$ with no edge between X and Z in the skeleton, if $X \not\perp\!\!\!\perp Z \mid \mathbf{S}$ for any valid $\mathbf{S}$ that includes Y
3. **Orientation Rules**: Exploits absence of v-structures, or the acyclicity constraint to orient further edges

Theorem (see SGS book) If P is **Markov** and **faithful** to a DAG $\mathcal{G}$, and we have a perfect **independence oracle**, then the SGS Algorithm recovers the **Markov equivalence class** of $\mathcal{G}$ (CPDAG).

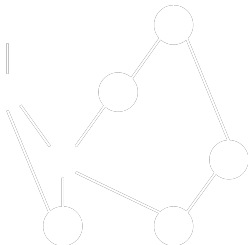
SGS Algorithm (Skeleton Phase)

Algorithm 2: SGS Algorithm

```
1  $\mathcal{H} \leftarrow$  complete undirected graph;  
  // Learn skeleton  
2 for  $A, B \in \mathbf{V}$ , where  $A \neq B$  do  
3   if  $\exists \mathbf{S} \subseteq \mathbf{V} \setminus \{A, B\} : A \perp\!\!\!\perp B \mid \mathbf{S}$  then  
4     Remove edge  $(A, B)$  from  $\mathcal{H}$  ;
```



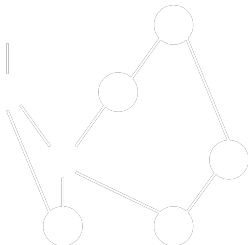
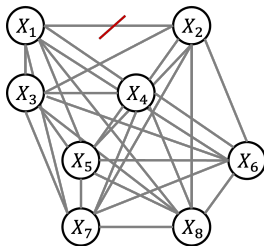
Initialize $\mathcal{H}$ as fully connected



SGS Algorithm (Skeleton Phase)

Algorithm 2: SGS Algorithm

```
1  $\mathcal{H} \leftarrow$  complete undirected graph;  
  // Learn skeleton  
2 for  $A, B \in \mathbf{V}$ , where  $A \neq B$  do  
3   if  $\exists \mathbf{S} \subseteq \mathbf{V} \setminus \{A, B\} : A \perp\!\!\!\perp B \mid \mathbf{S}$  then  
4     Remove edge  $(A, B)$  from  $\mathcal{H}$  ;
```

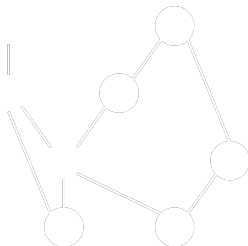
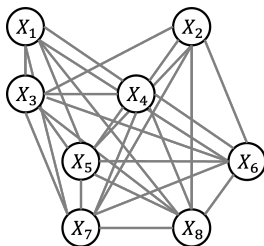


Find that $X_1 \perp\!\!\!\perp X_2$ $\stackrel{\text{Markov \& faithful}}{\Leftrightarrow} 1 \perp\!\!\!\perp_{\mathcal{G}} 2$.

SGS Algorithm (Skeleton Phase)

Algorithm 2: SGS Algorithm

```
1  $\mathcal{H} \leftarrow$  complete undirected graph;  
  // Learn skeleton  
2 for  $A, B \in \mathbf{V}$ , where  $A \neq B$  do  
3   if  $\exists \mathbf{S} \subseteq \mathbf{V} \setminus \{A, B\} : A \perp\!\!\!\perp B \mid \mathbf{S}$  then  
4     Remove edge  $(A, B)$  from  $\mathcal{H}$  ;
```

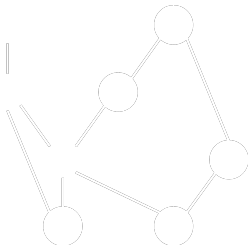
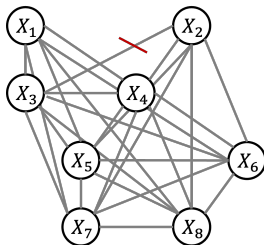


Find that $X_1 \perp\!\!\!\perp X_2 \stackrel{\text{Markov \& faithful}}{\Leftrightarrow} 1 \perp\!\!\!\perp_G 2$. Thus, **delete** the corresponding edge $1 - 2$.

SGS Algorithm (Skeleton Phase)

Algorithm 2: SGS Algorithm

```
1  $\mathcal{H} \leftarrow$  complete undirected graph;  
  // Learn skeleton  
2 for  $A, B \in \mathbf{V}$ , where  $A \neq B$  do  
3   if  $\exists \mathbf{S} \subseteq \mathbf{V} \setminus \{A, B\} : A \perp\!\!\!\perp B \mid \mathbf{S}$  then  
4     Remove edge  $(A, B)$  from  $\mathcal{H}$  ;
```

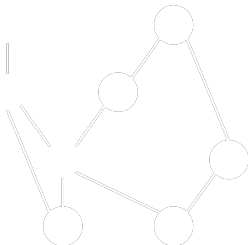
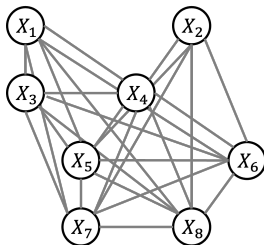


Find that $X_2 \perp\!\!\!\perp X_3$.

SGS Algorithm (Skeleton Phase)

Algorithm 2: SGS Algorithm

```
1  $\mathcal{H} \leftarrow$  complete undirected graph;  
  // Learn skeleton  
2 for  $A, B \in \mathbf{V}$ , where  $A \neq B$  do  
3   if  $\exists \mathbf{S} \subseteq \mathbf{V} \setminus \{A, B\} : A \perp\!\!\!\perp B \mid \mathbf{S}$  then  
4     Remove edge  $(A, B)$  from  $\mathcal{H}$  ;
```

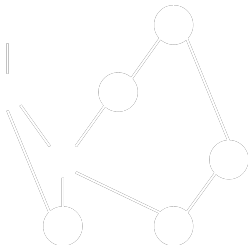
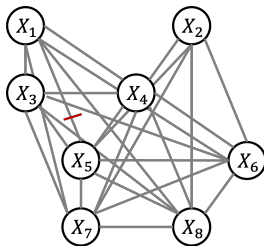


Find that $X_2 \perp\!\!\!\perp X_3$. Thus, **delete** the corresponding edge $2 - 3$.

SGS Algorithm (Skeleton Phase)

Algorithm 2: SGS Algorithm

```
1  $\mathcal{H} \leftarrow$  complete undirected graph;  
  // Learn skeleton  
2 for  $A, B \in \mathbf{V}$ , where  $A \neq B$  do  
3   if  $\exists \mathbf{S} \subseteq \mathbf{V} \setminus \{A, B\} : A \perp\!\!\!\perp B \mid \mathbf{S}$  then  
4     Remove edge  $(A, B)$  from  $\mathcal{H}$  ;
```

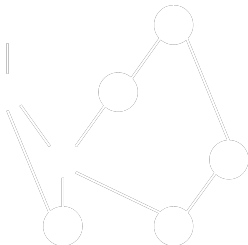
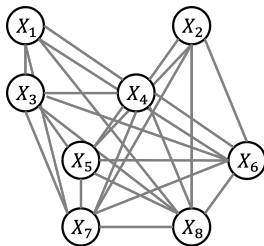


Find that $X_1 \perp\!\!\!\perp X_5 \mid X_3$.

SGS Algorithm (Skeleton Phase)

Algorithm 2: SGS Algorithm

```
1  $\mathcal{H} \leftarrow$  complete undirected graph;  
  // Learn skeleton  
2 for  $A, B \in \mathbf{V}$ , where  $A \neq B$  do  
3   if  $\exists \mathbf{S} \subseteq \mathbf{V} \setminus \{A, B\} : A \perp\!\!\!\perp B \mid \mathbf{S}$  then  
4     Remove edge  $(A, B)$  from  $\mathcal{H}$  ;
```

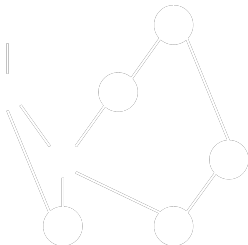
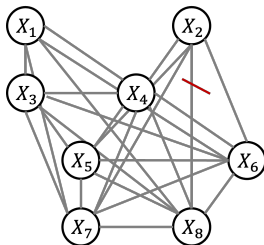


Find that $X_1 \perp\!\!\!\perp X_5 \mid X_3$. Thus, **delete** the corresponding edge $1 - 5$.

SGS Algorithm (Skeleton Phase)

Algorithm 2: SGS Algorithm

```
1  $\mathcal{H} \leftarrow$  complete undirected graph;  
  // Learn skeleton  
2 for  $A, B \in \mathbf{V}$ , where  $A \neq B$  do  
3   if  $\exists \mathbf{S} \subseteq \mathbf{V} \setminus \{A, B\} : A \perp\!\!\!\perp B \mid \mathbf{S}$  then  
4     Remove edge  $(A, B)$  from  $\mathcal{H}$  ;
```

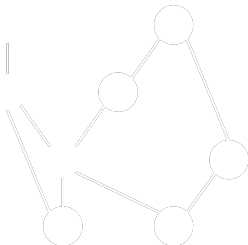
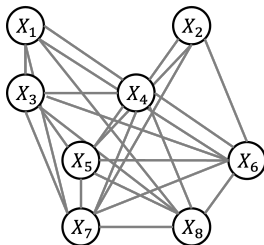


Find that $X_2 \perp\!\!\!\perp X_8 \mid \{X_4, X_6\}$.

SGS Algorithm (Skeleton Phase)

Algorithm 2: SGS Algorithm

```
1  $\mathcal{H} \leftarrow$  complete undirected graph;  
  // Learn skeleton  
2 for  $A, B \in \mathbf{V}$ , where  $A \neq B$  do  
3   if  $\exists \mathbf{S} \subseteq \mathbf{V} \setminus \{A, B\} : A \perp\!\!\!\perp B \mid \mathbf{S}$  then  
4     Remove edge  $(A, B)$  from  $\mathcal{H}$  ;
```

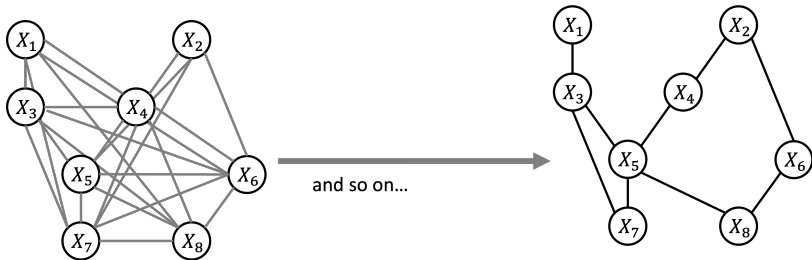


Find that $X_2 \perp\!\!\!\perp X_8 \mid \{X_4, X_6\}$. Thus, **delete** the corresponding edge $2 - 8$.

SGS Algorithm (Skeleton Phase)

Algorithm 2: SGS Algorithm

```
1  $\mathcal{H} \leftarrow$  complete undirected graph;  
  // Learn skeleton  
2 for  $A, B \in \mathbf{V}$ , where  $A \neq B$  do  
3   if  $\exists \mathbf{S} \subseteq \mathbf{V} \setminus \{A, B\} : A \perp\!\!\!\perp B \mid \mathbf{S}$  then  
4     Remove edge  $(A, B)$  from  $\mathcal{H}$  ;
```



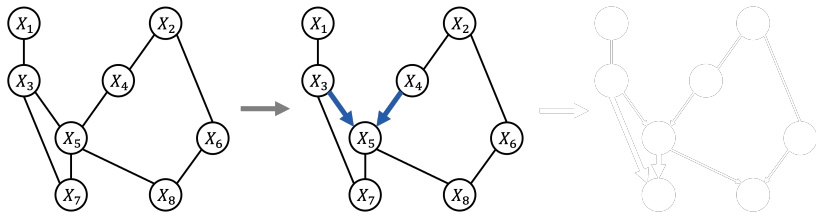
Recover the **full skeleton** by repeating this procedure for each pair.

SGS Algorithm (V-Structures)

Algorithm 2: SGS Algorithm

// Find v-structures

```
1 for all triples  $A - B - C \in \mathcal{H}$ , with  $A$  not adjacent to  $C$  do
2   if  $\forall \mathbf{S} \subseteq \mathbf{V} \setminus \{A, B, C\} : A \not\perp\!\!\!\perp C \mid \mathbf{S} \cup \{B\}$  then
3     Orient  $A - B - C$  as  $A \rightarrow B \leftarrow C$  in  $\mathcal{H}$ ;
```



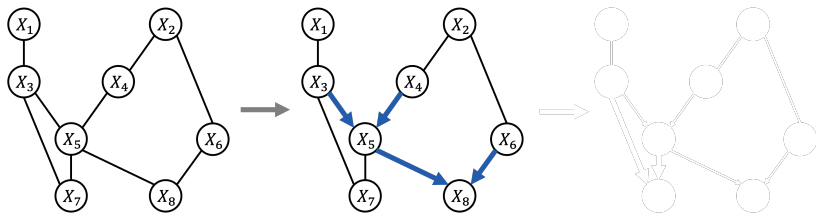
- $X_3 \perp\!\!\!\perp X_4$, we have $3 - 5 - 4$
- $X_3 \not\perp\!\!\!\perp X_4 \mid X_5 \cup \mathbf{S}$, for any $\mathbf{S} \subseteq \mathbf{V} \setminus \{X_3, X_4, X_5\}$
- Thus, orient $3 \rightarrow 5 \leftarrow 4$ in $\mathcal{H}$

SGS Algorithm (V-Structures)

Algorithm 2: SGS Algorithm

// Find v-structures

```
1 for all triples  $A - B - C \in \mathcal{H}$ , with  $A$  not adjacent to  $C$  do
2   if  $\forall \mathbf{S} \subseteq \mathbf{V} \setminus \{A, B, C\} : A \not\perp\!\!\!\perp C \mid \mathbf{S} \cup \{B\}$  then
3     Orient  $A - B - C$  as  $A \rightarrow B \leftarrow C$  in  $\mathcal{H}$ ;
```



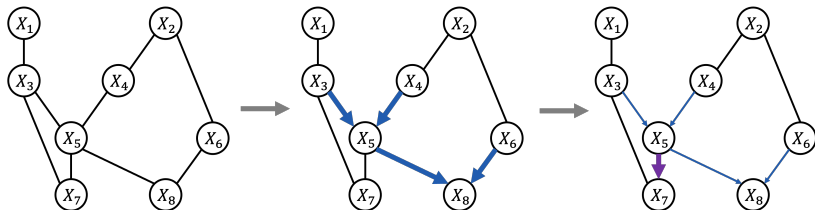
- $X_5 \perp\!\!\!\perp X_6 \mid X_4$, we have $5 - 8 - 6$
- $X_5 \not\perp\!\!\!\perp X_6 \mid X_8 \cup \mathbf{S}$, for any $\mathbf{S} \subseteq \mathbf{V} \setminus \{X_5, X_6, X_8\}$
- Thus, orient $5 \rightarrow 8 \leftarrow 6$ in $\mathcal{H}$

SGS Algorithm (Orientation Rules)

Algorithm 2: SGS Algorithm

// Apply orientation rules

```
1 repeat
2   if  $A \rightarrow B$ , and  $B - C$  in  $\mathcal{H}$ , and  $A$  and  $C$  are not adjacent in  $\mathcal{H}$  then
3     | Orient  $B - C$  as  $B \rightarrow C$  in  $\mathcal{H}$  (otherwise v-structure);
4   if  $\exists$  directed path from  $A$  to  $B$  in  $\mathcal{H}$  and an undirected edge  $A - B$  then
5     | Orient  $A - B$  as  $A \rightarrow B$  (otherwise cycle);
6 until no more edge can be oriented;
```



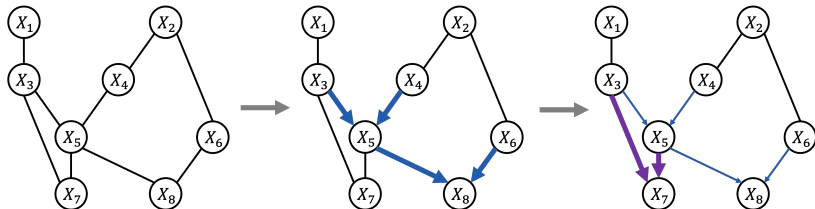
- $4 \rightarrow 5$, $5 - 7 \in \mathcal{H}$, and 7 is not adjacent to 4 in $\mathcal{H}$, but $X_4 \perp\!\!\!\perp X_7 \mid \{X_5, X_3\}$
- Hence, orient $5 - 7$ as $5 \rightarrow 7$

SGS Algorithm (Orientation Rules)

Algorithm 2: SGS Algorithm

// Apply orientation rules

```
1 repeat
2   if  $A \rightarrow B$ , and  $B - C$  in  $\mathcal{H}$ , and  $A$  and  $C$  are not adjacent in  $\mathcal{H}$  then
3     | Orient  $B - C$  as  $B \rightarrow C$  in  $\mathcal{H}$  (otherwise v-structure);
4   if  $\exists$  directed path from  $A$  to  $B$  in  $\mathcal{H}$  and an undirected edge  $A - B$  then
5     | Orient  $A - B$  as  $A \rightarrow B$  (otherwise cycle);
6 until no more edge can be oriented;
```



- $\exists$ path $3 \rightarrow 5 \rightarrow 7$ and edge $3 - 7 \in \mathcal{H}$
- Thus, orient $3 - 7$ as $3 \rightarrow 7$ to **avoid a cycle**

SGS Algorithm

Algorithm 2: SGS Algorithm

```
1  $\mathcal{H} \leftarrow$  complete undirected graph;
  // Learn skeleton
2 for  $A, B \in \mathbf{V}$ , where  $A \neq B$  do
3   if  $\exists \mathbf{S} \subseteq \mathbf{V} \setminus \{A, B\} : A \perp\!\!\!\perp B \mid \mathbf{S}$  then
4      $\perp$  Remove edge  $(A, B)$  from  $\mathcal{H}$  ;
  // Find v-structures
5 for all triples  $A - B - C \in \mathcal{H}$ , with  $A$  not adjacent to  $C$  do
6   if  $\forall \mathbf{S} \subseteq \mathbf{V} \setminus \{A, B, C\} : A \not\perp\!\!\!\perp C \mid \mathbf{S} \cup \{B\}$  then
7      $\perp$  Orient  $A - B - C$  as  $A \rightarrow B \leftarrow C$  in  $\mathcal{H}$ ;
  // Apply orientation rules
8 repeat
9   if  $A \rightarrow B$ , and  $B - C$  in  $\mathcal{H}$ , and  $A$  and  $C$  are not adjacent in  $\mathcal{H}$  then
10     $\perp$  Orient  $B - C$  as  $B \rightarrow C$  in  $\mathcal{H}$  (otherwise v-structure);
11   if  $\exists$  directed path from  $A$  to  $B$  in  $\mathcal{H}$  and and an undirected edge  $A - B$  then
12     $\perp$  Orient  $A - B$  as  $A \rightarrow B$  (otherwise cycle);
13 until no more edge can be oriented;
```

Assumptions:

- **Markov:** Weak assumption (standard for Bayesian networks)
- **Faithfulness:** We saw that violations exists but they are unlikely (at least in linear systems they have measure zero)
- **Independence oracle:** Can be a strong assumption especially when the number of nodes d is large (we need to condition on any possible subset; grows exponential with d)

Stability - Mistakes were made...

Assume there is a **mistake** in the **skeleton phase**:

1. $\cancel{\perp\!\!\!\perp}$ instead of $\perp\!\!\!\perp$: Might not be harmful as we potentially still delete the edge based on another conditional independence
2. $\perp\!\!\!\perp$ instead of $\cancel{\perp\!\!\!\perp}$: Edge will be **wrongly deleted**, no effect on remaining edges (but potentially the orientation phase)

Assume there is a **mistake** in the **orientation phases**:

- Can have a potentially large impact on the orientation of other edges

Stability with Faithfulness Violation

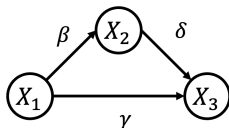
Recall the cancelling path?

Set $\gamma = -(\beta\delta)$ in the SCM ($N_i \sim \mathcal{N}(0, 1)$, indep.):

$$X_1 := N_1$$

$$X_2 := 0.5X_1 + N_2$$

$$X_3 := 2X_1 - 4X_2 + N_3$$



Question: Which graph would the SGS algorithm learn?

- **Skeleton:** $X_1 - X_2 - X_3$ $\nleftrightarrow$ ($X_1 - X_3$ missing)
- **V-structure:** X_1 and X_3 are not adjacent after the skeleton phase, so we have to check if $X_1 \not\perp\!\!\!\perp X_3 \mid X_2$, i.e., compute the partial correlation:

$$\text{Corr}(X_1 - E[X_1|X_2], X_3 - E[X_3|X_2]) > 0$$

and orient $X_1 - X_2 - X_3$ as $X_1 \rightarrow X_2 \leftarrow X_3$ $\nleftrightarrow$

Runtime Complexity

```
// Learn skeleton
1 for  $A, B \in V$ , where  $A \neq B$  do
2   if  $\exists S \subseteq V \setminus \{A, B\} : A \perp\!\!\!\perp B \mid S$  then
3     Remove edge  $(A, B)$  from  $\mathcal{H}$  ;
```

The **skeleton phase** has an **exponential search space** growing with the number of nodes d , for any pair of nodes.

The PC Algorithm, named after Peter Spirtes and Clark Glymour aims to improve upon this aspect!

Peter-Clark (PC) Algorithm

Modifies the skeleton search phase of the SGS Algorithm:

- **Restricts possible separating sets** to nodes adjacent to node A ($S \subseteq \text{Adj}(A) \setminus \{B\}$) in the ordered pair (A, B)
- Starts with separating sets of size 0 for all pairs and then **gradually increases** the maximum size k of the separating set
- When $k > |\text{Adj}(A) \setminus \{B\}|$ for all pairs, it stops

Whenever that graph is not fully connected, **PC is more efficient than the SGS algorithm!**

PC Algorithm (Skeleton Search)

Algorithm 3: PC Algorithm (Skeleton Search)

```
1  $\mathcal{H} \leftarrow$  complete undirected graph;
2  $k \leftarrow -1$  ;
3 repeat
4    $k \leftarrow k + 1$ ;
5   repeat
6     Pick a (new) pair  $(A, B)$  with  $k \leq |\text{Adj}(A) \setminus \{B\}|$  ;
7     repeat
8       Chose (new) set  $\mathbf{S} \subseteq \text{Adj}(A) \setminus \{B\}$ ,  $|\mathbf{S}| = k$  ;
9       if  $A \perp\!\!\!\perp B \mid \mathbf{S}$  then
10         Remove edge  $(A, B)$  from  $\mathcal{H}$  ;
11         Store separating set  $\text{sepset}(A, B) = \text{sepset}(B, A) = \mathbf{S}$ 
12       until edge  $(A, B) \notin \mathcal{H}$  or all  $\mathbf{S}$  of size  $k$  have been considered;
13   until all ordered pairs  $(A, B) \in \mathcal{H}$  with  $k \leq |\text{Adj}(A) \setminus \{B\}|$  have been considered;
14 until all pairs  $(A, B) \in \mathcal{H}$  satisfy  $k > |\text{Adj}(A) \setminus \{B\}|$ ;
```

PC Algorithm (V-Structures and Orientation)

```
// Orient v-structures
1 for all triples  $A - B - C \in \mathcal{H}$ , with  $A$  not adjacent to  $C$  do
2   if  $B \notin \text{sepset}(A, C)$  then
3     Orient  $A - B - C$  as  $A \rightarrow B \leftarrow C$  in  $\mathcal{H}$ ;
```

// Orientation phase

```
4 Apply the same additional rules as for the SGS algorithm
```

Notice how the orientation of v-structures is **merely a lookup** from the skeleton search.

The worst case **runtime complexity** is

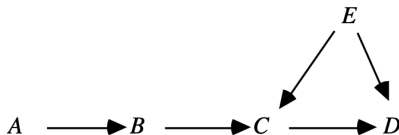
$$2 \binom{d}{2} \sum_{i=0}^k \binom{d-1}{i},$$

with maximum indegree k , and number of nodes d .

Stability of PC

Both the skeleton phase and the orientation phase are **not very stable** when mistakes are made in a finite sample.

- As in SGS, **errors** in the **orientation phase** can **accumulate**
- For the skeleton phase, it is **important that the neighbours are correct**, see example from the SGS book below:



- Assume the edge $E - D$ is **mistakenly deleted**
- Then, $B - D$ will **not be removed** in the following steps

PC Algorithm - Sample Version

- Instead of a conditional independence “oracle”, we **perform conditional independence tests**
- In the **multivariate Gaussian** setting, this is equivalent to testing for zero **partial correlation**:

$H_0 : \rho_{X,Y|Z} = 0$ versus $H_A : \rho_{X,Y|Z} \neq 0$, with

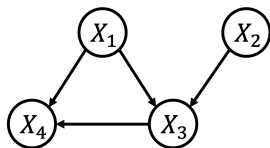
$$\rho_{X,Y|Z} = \frac{\rho_{X,Y} - \rho_{X,Z}\rho_{Z,Y}}{\sqrt{(1 - \rho_{X,Z}^2)(1 - \rho_{Y,Z}^2)}}$$

- Note that $\rho_{X,Y|Z}$ equals the **correlation between the residuals** after linearly regressing X on Z and Y on Z .
- The significance level α serves as a tuning parameter for the PC algorithm, and can heavily influence the result.

PC - Question

Consider the DAG $\mathcal{G}$ on the right and suppose that:

- We are given observations of variables (X_1, X_2, X_3, X_4)
- The assumptions of the PC algorithm are fulfilled
- We have access to an independence oracle (perfect independence test)

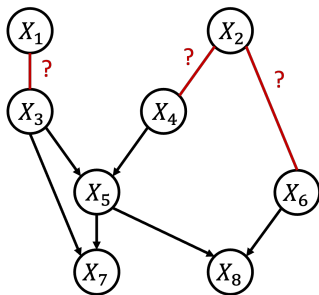


TRUE or **FALSE**?

- If the data come from $\mathcal{G}$, the edge between X_1 and X_2 will be removed first in the skeleton phase.
- The Markov equivalence class of $\mathcal{G}$ only contains one DAG.

What's next?

How can we go *beyond* the Markov equivalence class?



- **Recommended reading** Chapter 5 in *Causation, Prediction, and Search* by Spirtes, Glymour and Scheines (2000)
- This course *builds upon and extends* the material from “*Causality*” (WiSe 2022/23) that was kindly provided by **Stefan Harmeling**
- This lecture, *builds upon and extends* the slides of the course “*Causality*” (2021) by Christina Heinze-Deml

Thank you!

Lecture



Exercise

